

# Image Processing

## Assignment – Edge Detection Using the Sobel Filter

### Part A – Manual Computation

Alphin Joseph Alex  
15 S2 ME  
Page No. \_\_\_\_\_  
Date / /

### Edge Detection Using Sobel Filter

Given input image size :  $5 \times 5$

|    |    |    |    |    |
|----|----|----|----|----|
| 10 | 10 | 10 | 80 | 80 |
| 10 | 10 | 10 | 80 | 80 |
| 10 | 10 | 10 | 80 | 80 |
| 10 | 10 | 10 | 80 | 80 |
| 10 | 10 | 10 | 80 | 80 |

Sobel  $X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

1) Formula =  $N - k + 1$  [more specifically  $\frac{N - k + 2P}{s} + 1$ ]

Output Size =  $\frac{5 - 3 + 2(0)}{1} + 1 = 3$

Output size =  $3 \times 3$

$N \rightarrow$  input image size  
 $k \rightarrow$  Filter size  
 $P \rightarrow$  padding  
 $s \rightarrow$  stride

2) First Output (top-left)

Image Patch

|    |    |    |
|----|----|----|
| 10 | 10 | 10 |
| 10 | 10 | 10 |
| 10 | 10 | 10 |

Sobel  $X$  kernel

|    |   |   |
|----|---|---|
| -1 | 0 | 1 |
| -2 | 0 | 2 |
| -1 | 0 | 1 |

Page No.

Date

|    |    |    |    |    |
|----|----|----|----|----|
| 10 | 10 | 10 | 80 | 80 |
| 10 | 10 | 10 | 80 | 80 |
| 10 | 10 | 10 | 80 | 80 |
| 10 | 10 | 10 | 80 | 80 |
| 10 | 10 | 10 | 80 | 80 |

That box in the matrix (image size matrix) shows the kernel placement.

|          |                  |                 |                 |
|----------|------------------|-----------------|-----------------|
| Output = | 10 <sup>-1</sup> | 10 <sup>0</sup> | 10 <sup>1</sup> |
|          | 10 <sup>-2</sup> | 10 <sup>0</sup> | 10 <sup>2</sup> |
|          | 10 <sup>-1</sup> | 10 <sup>0</sup> | 10 <sup>1</sup> |

$$\begin{aligned}
 &= 10(-1) + 10(0) + 10(1) + \\
 &\quad 10(-2) + 10(0) + 10(2) + \\
 &\quad 10(-1) + 10(0) + 10(1) \\
 &= -10 + 0 + 10 - 20 + 0 + 20 - 10 + 0 + 10 \\
 &= 0
 \end{aligned}$$

Convolution = 0

$$\begin{aligned}
 3) \quad \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix} &= (-1)(10) + (1)(80) + (-2)(10) + (0)(10) + \\
 &\quad (2)(80) + (-1)(10) + (0)(10) + (1)(80) \\
 &= -10 + 80 - 20 + 160 - 10 + 80 \\
 &= 280
 \end{aligned}$$

Page No. \_\_\_\_\_

Date     /     /

$$\begin{bmatrix} 10 & 80 & 80 \\ 10 & 80 & 80 \\ 10 & 80 & 80 \end{bmatrix} = (-1)(10) + (0)(80) + (1)(80) + (-2)(10) + (0)(80) + (2)(80) + (-1)(10) + (0)(80) + (1)(80) \\ = -10 + 80 - 20 + 160 - 10 + 80 \\ = 280$$

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix} = 0 \quad \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix} = 280 \quad \begin{bmatrix} 10 & 80 & 80 \\ 10 & 80 & 80 \\ 10 & 80 & 80 \end{bmatrix} = 280$$

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix} = 0 \quad \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix} = 280 \quad \begin{bmatrix} 10 & 80 & 80 \\ 10 & 80 & 80 \\ 10 & 80 & 80 \end{bmatrix} = 280$$

3) Complete output matrix =  $\begin{bmatrix} 0 & 280 & 280 \\ 0 & 280 & 280 \\ 0 & 280 & 280 \end{bmatrix}$

e) Analysis

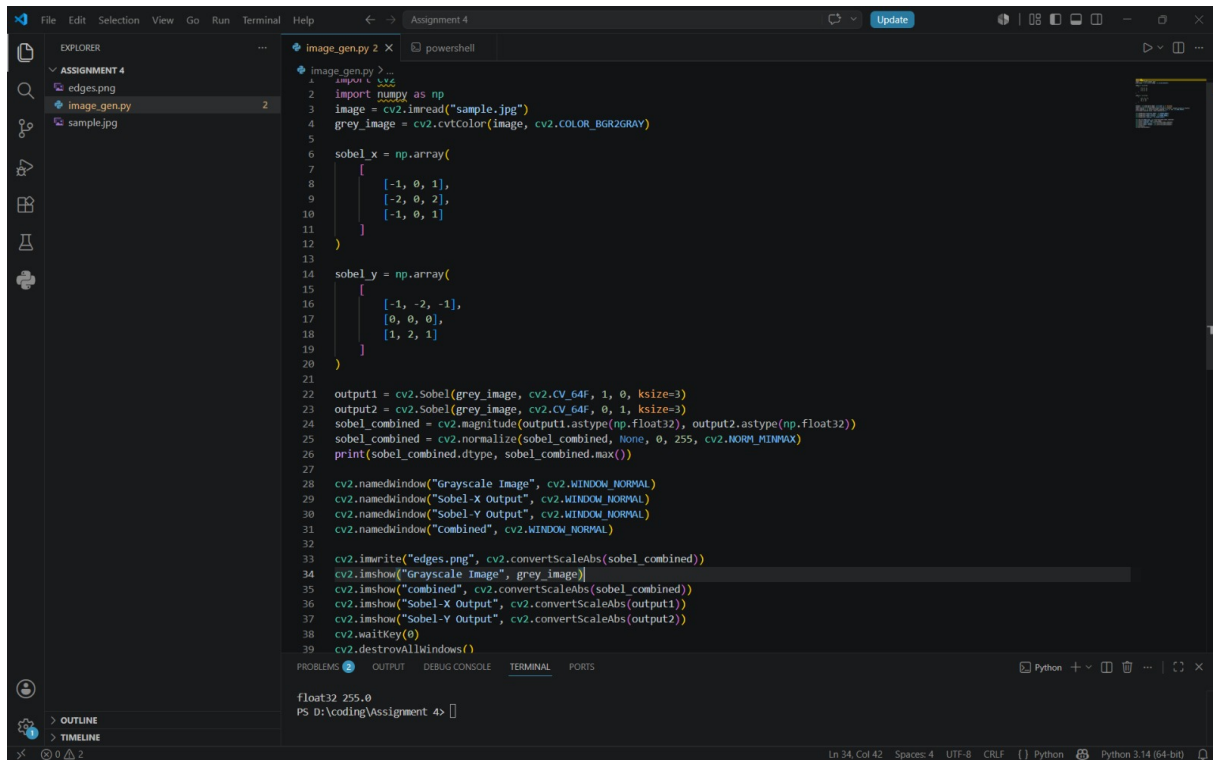
- The large values (280) indicate the presence of a vertical edge
- The edge occurs where the pixel values change from 10 to 80
- Homogenous regions produce values close to zero because there is little or no intensity change between neighbouring pixels
- The Sobel operator responds strongly only where there is a sudden change in intensity

## Part B – Programming Using OpenCV

### Python Source Code

```
import cv2
import numpy as np
image = cv2.imread("sample.jpg")
grey_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
sobel_x = np.array(
    [
        [-1, 0, 1],
        [-2, 0, 2],
        [-1, 0, 1]
    ]
)
sobel_y = np.array(
    [
        [-1, -2, -1],
        [0, 0, 0],
        [1, 2, 1]
    ]
)
output1 = cv2.Sobel(grey_image, cv2.CV_64F, 1, 0, ksize=3)
output2 = cv2.Sobel(grey_image, cv2.CV_64F, 0, 1, ksize=3)
sobel_combined = cv2.magnitude(output1.astype(np.float32), output2.astype(np.float32))
sobel_combined = cv2.normalize(sobel_combined, None, 0, 255, cv2.NORM_MINMAX)
print(sobel_combined.dtype, sobel_combined.max())
cv2.namedWindow("Grayscale Image", cv2.WINDOW_NORMAL)
cv2.namedWindow("Sobel-X Output", cv2.WINDOW_NORMAL)
cv2.namedWindow("Sobel-Y Output", cv2.WINDOW_NORMAL)
cv2.namedWindow("Combined", cv2.WINDOW_NORMAL)
cv2.imwrite("edges.png", cv2.convertScaleAbs(sobel_combined))
cv2.imshow("Grayscale Image", grey_image)
cv2.imshow("Combined", cv2.convertScaleAbs(sobel_combined))
cv2.imshow("Sobel-X Output", cv2.convertScaleAbs(output1))
cv2.imshow("Sobel-Y Output", cv2.convertScaleAbs(output2))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

## Code Screenshot (VS Code)



The screenshot shows the Visual Studio Code interface with a Python file named `image_gen.py` open. The code implements Sobel edge detection using OpenCV and NumPy. It reads an image, converts it to grayscale, and calculates the Sobel gradients in the x and y directions. The results are displayed in four separate windows: 'Grayscale Image', 'Sobel-X Output', 'Sobel-Y Output', and 'Combined'. The terminal at the bottom shows the output of the program, indicating the data type and maximum value of the combined Sobel result.

```
1 import cv2
2 import numpy as np
3 image = cv2.imread("sample.jpg")
4 grey_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
5
6 sobel_x = np.array(
7     [
8         [-1, 0, 1],
9         [-2, 0, 2],
10        [-1, 0, 1]
11     ]
12 )
13
14 sobel_y = np.array(
15     [
16         [-1, -2, -1],
17         [0, 0, 0],
18         [1, 2, 1]
19     ]
20 )
21
22 output1 = cv2.Sobel(grey_image, cv2.CV_64F, 1, 0, ksize=3)
23 output2 = cv2.Sobel(grey_image, cv2.CV_64F, 0, 1, ksize=3)
24 sobel_combined = cv2.magnitude(output1.astype(np.float32), output2.astype(np.float32))
25 sobel_combined = cv2.normalize(sobel_combined, None, 0, 255, cv2.NORM_MINMAX)
26 print(sobel_combined.dtype, sobel_combined.max())
27
28 cv2.namedWindow("Grayscale Image", cv2.WINDOW_NORMAL)
29 cv2.namedWindow("Sobel-X Output", cv2.WINDOW_NORMAL)
30 cv2.namedWindow("Sobel-Y Output", cv2.WINDOW_NORMAL)
31 cv2.namedWindow("Combined", cv2.WINDOW_NORMAL)
32
33 cv2.imwrite("edges.png", cv2.convertScaleAbs(sobel_combined))
34 cv2.imshow("Grayscale Image", grey_image)
35 cv2.imshow("combined", cv2.convertScaleAbs(sobel_combined))
36 cv2.imshow("Sobel-X Output", cv2.convertScaleAbs(output1))
37 cv2.imshow("Sobel-Y Output", cv2.convertScaleAbs(output2))
38 cv2.waitKey(0)
39 cv2.destroyAllWindows()
```

float32 255.0  
PS D:\coding\Assignment 4>

Figure 0: Python source code in VS Code



## Screenshots – Program Output Windows

### Original Image (sample.jpg)

---



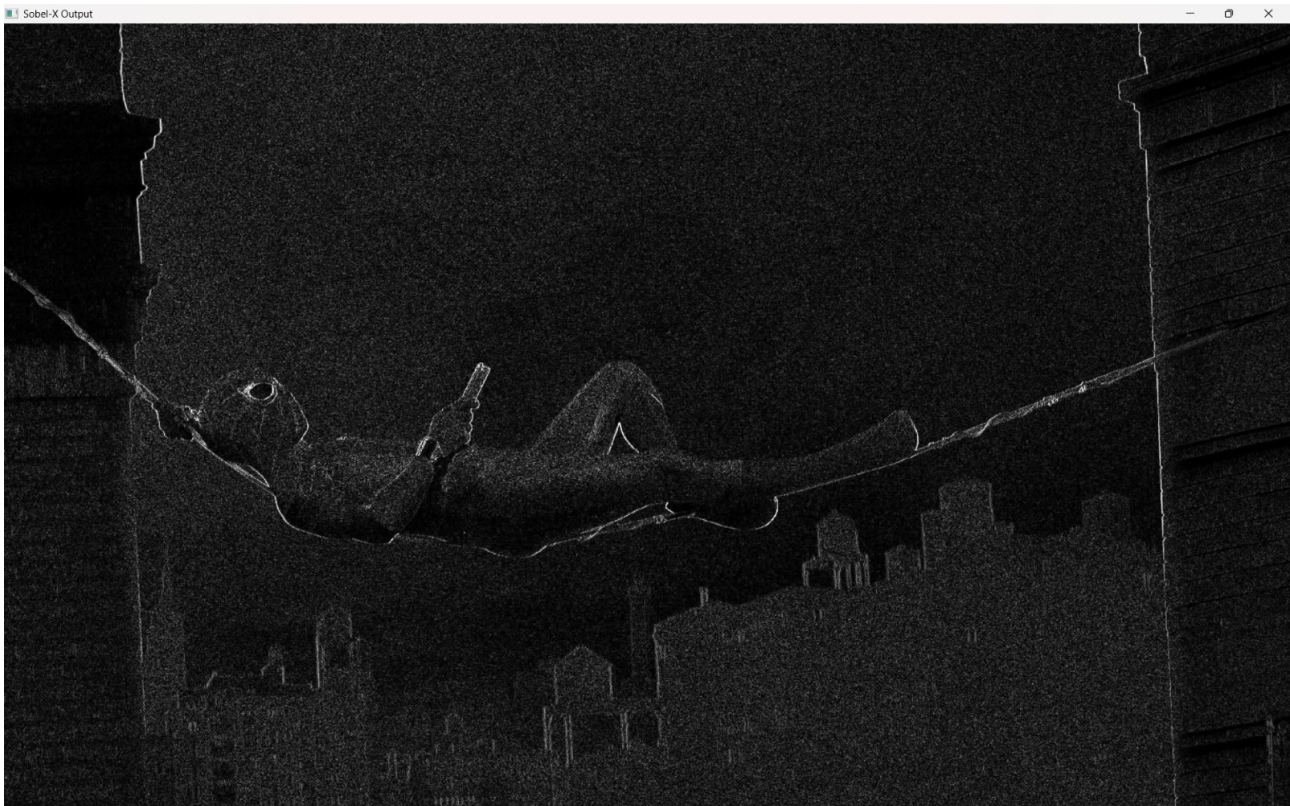
Figure 1: Original color input image (sample.jpg)

## Grayscale Image



Figure 2: Image after BGR  $\rightarrow$  Grayscale conversion

## Sobel-X Output



*Figure 3: Sobel-X operator output – detects vertical edges*



## Sobel-Y Output



*Figure 4: Sobel-Y operator output – detects horizontal edges*

### Final Edge-Detected Output (edges.png)

---



Figure 5: Combined gradient magnitude, normalized to [0-255]